

The logo for ENSTA (École Nationale Supérieure de Techniques Avancées) is displayed in blue. It consists of the letters "ENSTA" in a bold, sans-serif font, with a stylized "2" that incorporates a circular element.

ECOLE NATIONALE SUPÉRIEURE DE TECHNIQUES
AVANCÉES

ENSTA 2E ANNÉE

COURS APM 4RO03

LightUp

Serena DAGHER Yara EL CHAM

JEUX ET R.O

8 Mai 2026

Lien Code source

Table des matières

1	Présentation du jeu <i>Light Up</i>	3
1.1	Règles du jeu	3
1.2	Format du fichier d'instance	3
2	Formulation mathématique du problème	3
2.1	Vue d'ensemble	3
2.2	Variables de décision	4
2.3	Fonction objectif	4
2.4	Contrainte 1 : chaque case blanche doit être éclairée	4
2.5	Contrainte 2 : deux lampes ne peuvent pas se voir	5
2.6	Contrainte 3 : cases noires numérotées	5
2.7	Résumé du modèle PLNE	6
3	Implémentation en Julia	6
3.1	Structure du projet	6
3.2	Lecture et affichage (<code>io.jl</code>)	6
3.3	Résolution CPLEX (<code>resolution.jl</code>)	7
3.4	Génération d'instances (<code>generation.jl</code>)	7
4	Exemple concret	7
4.1	Instance de test	7
4.2	Vérification sur le site officiel	8
5	Résultats	9
5.1	Résultats sur le dataset généré	9
5.2	Graphiques	9
5.3	Analyse des résultats	10
6	Test sur de vraies instances du jeu	11
6.1	Limites de la génération aléatoire pour les grandes grilles	11
6.2	Résultats sur quatre instances réelles	11
6.2.1	Instance 15×15 — 0.024s	12
6.2.2	Instance 20×20 — 0.025s	12
6.2.3	Instance 35×35 — 0.069s	13
6.2.4	Instance 50×50 — 0.038s	13
6.3	Comparaison des temps de résolution	14
6.4	Analyse	14
6.5	Impact de la densité de cases noires sur le temps de résolution	15

1 Présentation du jeu *Light Up*

1.1 Règles du jeu

Light Up (difficulté 3) est un puzzle logique sur une grille $n \times m$. Il y a deux types de cases :

- **Cases blanches** : cases libres où on peut placer une lampe.
- **Cases noires** : cases qui bloquent la lumière. Certaines portent un chiffre (0 à 4) indiquant le nombre exact de lampes adjacentes requises.

Une lampe éclaire toutes les cases blanches sur sa ligne et sa colonne, jusqu'à une case noire ou le bord. Une solution valide doit respecter trois règles :

1. Toute case blanche est éclairée par au moins une lampe.
2. Deux lampes ne peuvent pas se voir directement.
3. Chaque case noire numérotée est adjacente à exactement le nombre de lampes indiqué.

1.2 Format du fichier d'instance

Chaque case est encodée dans un fichier texte (une ligne par ligne de grille, cases séparées par des virgules) :

- "." : case blanche ;
- "N" : case noire sans contrainte ;
- "0" à "4" : case noire numérotée.

L'instance de test `instanceTest.txt` utilisée :

```
., ., ., ., .  
., ., 2, ., .  
., 3, ., 4, .  
., ., N, ., .  
., ., ., ., .
```

2 Formulation mathématique du problème

2.1 Vue d'ensemble

On modélise *Light Up* comme un problème de **Programmation Linéaire en Nombres Entiers (PLNE)**. On cherche juste une solution réalisable, donc il n'y a pas de vraie fonction objectif : on minimise 0.

2.2 Variables de décision

Soit V l'ensemble des cases blanches. Pour chaque case $v \in V$, on définit :

$$x_v = \begin{cases} 1 & \text{si une lampe est placée en } v, \\ 0 & \text{sinon.} \end{cases}$$

Les cases noires sont fixées à 0 directement dans le modèle.

```
1 @variable(m_model, x[1:n, 1:m], Bin)
2 for i in 1:n, j in 1:m
3     if isBlack(grid, i, j)
4         @constraint(m_model, x[i, j] == 0) # cases noires = pas de lampe
5     end
6 end
```

2.3 Fonction objectif

$$\min \quad 0$$

2.4 Contrainte 1 : chaque case blanche doit être éclairée

Pour chaque case blanche c , on définit $L(c)$ comme l'ensemble des cases visibles depuis c sur sa ligne et colonne (sans case noire entre elles). La contrainte est :

$$\forall c \in V, \quad \sum_{v \in L(c)} x_v \geq 1$$

La fonction `visibleFrom` explore les 4 directions depuis c et s'arrête à chaque case noire ou bord. On montre ici l'exploration vers le haut ; les trois autres directions suivent le même schéma :

```
1 function visibleFrom(grid, n, m, i0, j0)
2     visible = Set{Tuple{Int,Int}}()
3     push!(visible, (i0, j0)) # la case elle-meme est visible
4     for i in (i0-1):-1:1 # exploration vers le haut
5         isBlack(grid, i, j0) && break # arret si case noire
6         push!(visible, (i, j0))
7     end
8     # ... idem pour bas, gauche, droite
9     return visible
10 end
```

La contrainte est ensuite ajoutée pour chaque case blanche :

```

1 for i in 1:n, j in 1:m
2     if isWhite(grid, i, j)
3         L = visibleFrom(grid, n, m, i, j)
4         @constraint(m_model, sum(x[vi, vj] for (vi, vj) in L) >= 1)
5     end
6 end

```

2.5 Contrainte 2 : deux lampes ne peuvent pas se voir

Si $v \in L(u)$ avec $v \neq u$, alors u et v se voient directement. On réutilise `visibleFrom`, pas besoin d'une fonction supplémentaire. La contrainte s'écrit :

$$\forall u \in V, \forall v \in L(u), v \neq u, \quad x_u + x_v \leq 1$$

On stocke les paires déjà traitées pour éviter les doublons ((u, v) et (v, u) sont la même contrainte) :

```

1 added_pairs = Set{Tuple{Tuple{Int,Int}, Tuple{Int,Int}}}()
2 for i in 1:n, j in 1:m
3     if isWhite(grid, i, j)
4         for (vi, vj) in visibleFrom(grid, n, m, i, j)
5             if (vi, vj) != (i, j)
6                 # on trie la paire pour ne pas l'ajouter deux fois
7                 pair = (i,j) <= (vi,vj) ? ((i,j),(vi,vj)) : ((vi,vj),(i,j))
8                 if !(pair in added_pairs)
9                     push!(added_pairs, pair)
10                    @constraint(m_model, x[i,j] + x[vi,vj] <= 1)
11                end
12            end
13        end
14    end
15 end

```

2.6 Contrainte 3 : cases noires numérotées

Pour chaque case noire de valeur k_b , on note $N(b)$ ses voisins blancs. On impose :

$$\forall b \text{ numérotée}, \quad \sum_{v \in N(b)} x_v = k_b$$

Dans l'implémentation, on commence par récupérer les voisins blancs de la case noire numérotée. Si la case possède au moins un voisin blanc, on ajoute la contrainte imposant que la somme des lampes adjacentes soit égale au chiffre indiqué.

```

1 if !isempty(neighbors)
2   @constraint(m_model, sum(x[ni,nj] for (ni,nj) in neighbors) == kb)
3 end

```

2.7 Résumé du modèle PLNE

min 0
sous les contraintes :

$$\sum_{v \in L(c)} x_v \geq 1 \quad \forall c \in V \quad (\text{éclairage}) \quad (1)$$

$$x_u + x_v \leq 1 \quad \forall u \in V, \forall v \in L(u), v \neq u \quad (\text{pas de lampes qui se voient}) \quad (2)$$

$$\sum_{v \in N(b)} x_v = k_b \quad \forall b \text{ numérotée} \quad (\text{contrainte numérotée}) \quad (3)$$

$$x_v \in \{0, 1\} \quad \forall v \in V \quad (4)$$

Toute solution entière satisfaisant ces contraintes est une solution valide du puzzle.

3 Implémentation en Julia

3.1 Structure du projet

Le projet est organisé en quatre fichiers :

- `main.jl` : point d'entrée avec les fonctions de test.
- `src/io.jl` : lecture des instances (`readInputFile`), affichage (`displayGrid`, `displaySolution`).
- `src/resolution.jl` : modèle PLNE CPLEX (`cplexSolve`), résolution du dataset (`solveDataSet`).
- `src/generation.jl` : génération aléatoire d'instances (`generateInstance`, `generateDataSet`).

On utilise **JuMP** pour modéliser le PLNE et **CPLEX** comme solveur exact.

3.2 Lecture et affichage (`io.jl`)

`readInputFile` lit le fichier ligne par ligne et retourne la matrice `grid` avec ses dimensions `n` et `m`. `displaySolution` affiche `L` pour une lampe et `.` pour une case éclairée sans lampe.

3.3 Résolution CPLEX (resolution.jl)

`cplexSolve(n, m, grid)` construit et résout le modèle PLNE. Elle retourne le triplet (`SolutionFound`, `solveTime`, `x_val`). Les sorties de CPLEX sont désactivées (`CPX_PARAM_SCRIND` = 0).

3.4 Génération d'instances (generation.jl)

`generateInstance` génère une grille en deux passes séparées :

1. Placement des cases noires : chaque case devient noire avec probabilité `blackRatio` (défaut : 0.30).
2. Numérotation : chaque case noire est numérotée avec probabilité `numberedRatio` (défaut : 0.50), avec un chiffre entre 0 et son nombre de voisins blancs à *ce moment-là*.

Les deux passes sont volontairement séparées. Si on numérotait chaque case noire au moment de la placer, certains de ses voisins blancs pourraient devenir noirs ensuite, rendant son chiffre impossible à satisfaire. En numérotant après avoir tout placé, on garantit la cohérence de chaque contrainte.

`generateDataSet()` génère 20 instances pour chaque taille parmi {4, 5, 6, 7, 8, 10, 12, 15}, soit 160 instances au total.

Même avec cette précaution, certaines instances restent infaisables à cause de contradictions entre cases noires voisines. Par exemple, dans cette instance :

```
., 3, 0, ., 3, ., N, .      # ligne 7 : contradiction entre (7,3) et (7,5)
```

La case 0 en (7,3) interdit toute lampe en (7,4). Mais la case 3 en (7,5) n'a que 3 voisins blancs et les exige tous comme lampes, dont (7,4). C'est une contradiction directe. Ce problème est inévitable avec une génération purement aléatoire : les chiffres sont choisis localement, sans vérifier la compatibilité globale entre cases voisines.

4 Exemple concret

4.1 Instance de test

On teste l'instance `instanceTest.txt` (grille 5×5) :

```
julia> include("main.jl")
julia> testSolve()

=== Test cplexSolve ===
SolutionFound: true   |   Time: 0.011s
```



```

Solution:
+-----+
| . L . . . |
| . . 2 L . |
| L 3 L 4 L |
| . L # L . |
| . . L . . |
+-----+

```

CPLEX trouve une solution en 11 ms. On vérifie les trois règles :

- **Règle 1** : toutes les cases blanches sont éclairées.
- **Règle 2** : aucune lampe n'en voit une autre directement.
- **Règle 3** : la case 2 a 2 lampes adjacentes, la case 3 en a 3, la case 4 en a 4, et la case 0 en a 0.

4.2 Vérification sur le site officiel

On a vérifié la solution sur le site chiark.greenend.org.uk/~sgtatham/puzzles/.

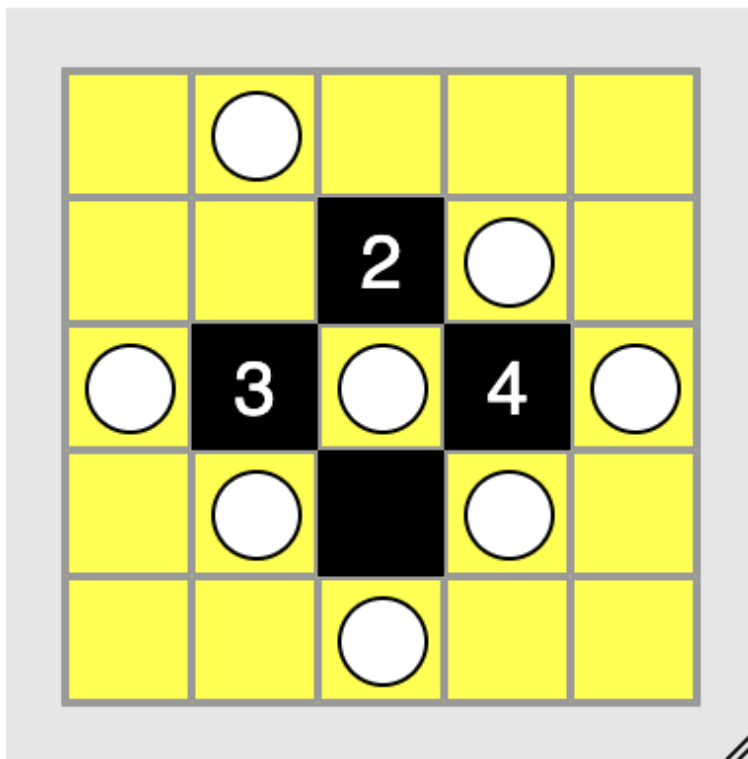


FIGURE 1 – Solution de l'instance test vérifiée sur le site officiel

5 Résultats

5.1 Résultats sur le dataset généré

On a généré 20 instances par taille et les a résolues avec CPLEX. Les résultats sont résumés dans le tableau suivant :

Taille	Instances	Solutions trouvées	Temps moyen (s)	Temps max (s)
<code>instanceTest.txt</code>	1	1	0.011	0.011
4×4	20	12	0.010	0.015
5×5	20	8	0.009	0.021
6×6	20	8	0.008	0.010
7×7	20	6	0.008	0.012
8×8	20	1	0.010	0.021
10×10	20	2	0.009	0.018
12×12	20	0	0.009	0.022
15×15	20	0	0.012	0.037

TABLE 1 – Résultats de la résolution CPLEX sur le dataset généré

5.2 Graphiques

La figure 2 montre le taux de faisabilité et les temps de résolution en fonction de la taille de la grille.

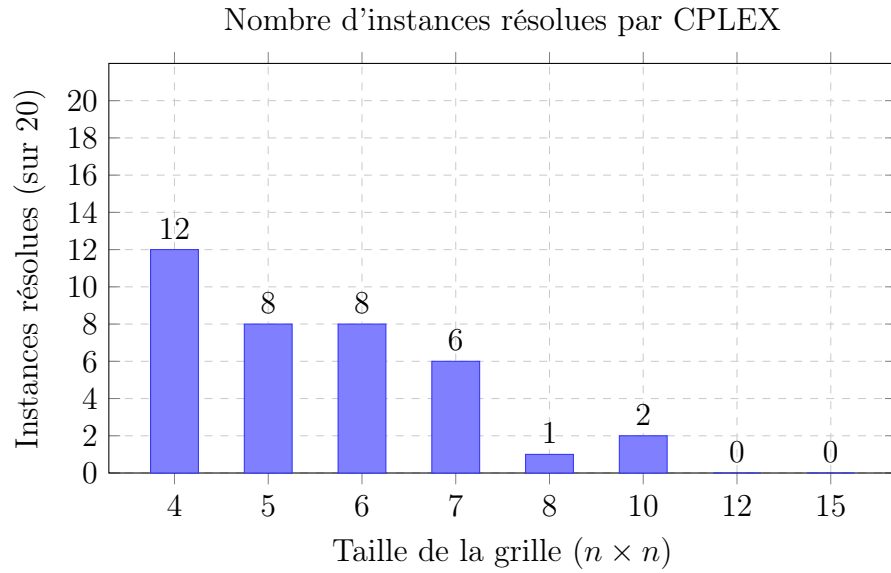


FIGURE 2 – Nombre d'instances faisables par taille de grille (sur 20 instances)

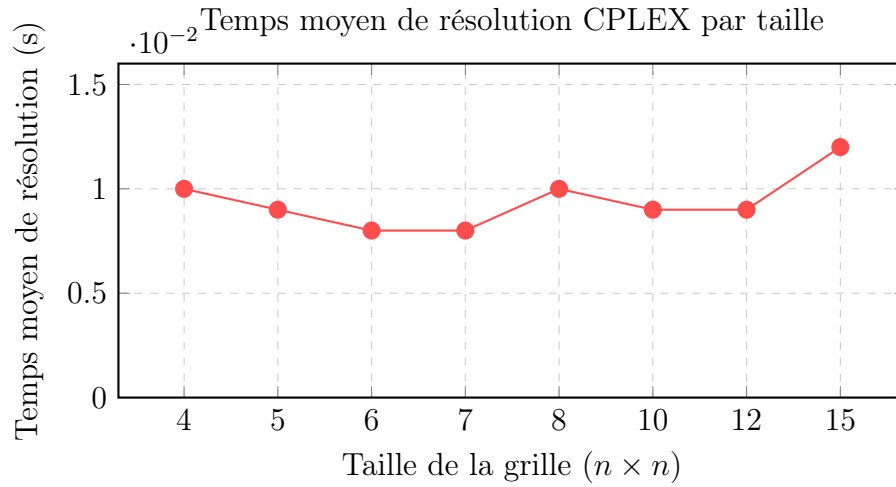


FIGURE 3 – Temps moyen de résolution en fonction de la taille de la grille

5.3 Analyse des résultats

Temps de calcul. Les temps de résolution sont très faibles pour toutes les tailles testées (inférieurs à 0.04s même en 15×15). CPLEX détecte l'infaisabilité très rapidement ce qui explique ces temps courts. On observe une légère augmentation pour les très grandes grilles (15×15 , max 0.037s), attendue vu la taille du modèle.

Solution toujours trouvée ? Non. Pour les instances générées aléatoirement, une grande part est infaisable : le générateur crée parfois des contraintes contradictoires entre cases noires numérotées voisines (par exemple une case 0 et une case 3 partageant un voisin blanc commun). En revanche, pour les instances faisables testées, comme `instanceTest.txt`, CPLEX trouve une solution valide respectant les règles du jeu.

Jusqu'à quelle taille ? Dans ces tests, la taille de la grille n'est pas vraiment le problème. Le problème vient surtout du fait que les instances générées aléatoirement deviennent souvent infaisables quand la taille augmente.

Cela ne veut donc pas dire que CPLEX ne peut pas résoudre de grandes grilles. Pour vérifier cela, on teste dans la section suivante de grandes instances réelles et faisables, prises depuis le site du jeu.

Gap après 10 minutes. Cette question ne s'applique pas ici : notre fonction objectif est constante (min 0), car Light Up est un problème de *faisabilité pure* — il n'y a pas de valeur à optimiser, uniquement une solution réalisable à trouver.

6 Test sur de vraies instances du jeu

6.1 Limites de la génération aléatoire pour les grandes grilles

Comme le montre le tableau 1, aucune des instances générées en 12×12 ou 15×15 n'est faisable. Ce n'est pas un problème de solveur : CPLEX prouve l'infaisabilité en quelques millisecondes. Le problème vient du générateur aléatoire, qui crée fréquemment des contradictions entre cases noires voisines. Pour valider notre solveur sur de vraies grandes grilles, nous avons encodé des instances directement issues du jeu officiel (chiark.greenend.org.uk/~sgtatham/puzzles/).

6.2 Résultats sur quatre instances réelles

Pour chaque instance, la solution retournée par CPLEX est vérifiée automatiquement à l'aide d'une fonction de validation qui contrôle les trois règles du jeu.

Les captures d'écran du site officiel sont ajoutées uniquement à titre illustratif, afin de visualiser les solutions obtenues sur les grandes grilles.

6.2.1 Instance 15×15 — 0.024s

```
julia> testSolve()

=== Test cplexSolve ===
Optimal: true | Time: 0.024s

Solution:
+-----+
| . 1 L . . # . . L 2 . L . 0 1 |
| L # 2 . . L . 0 . L . . . 1 L |
| # . L . . 2 L . . . . . 1 . . |
| . L . 1 L . . . # . 0 . L . . |
| . . 1 # # # # . . . . . . L . |
| . . L . . . . . . . . . . 0 . # |
| L 1 . . L # 0 . L 2 L . # . 0 |
| . . . . 2 . . L . . . # . L . . |
| 0 . 0 . L 2 L . 0 # L . . 0 . |
| # L 3 L . . . . . . . . . . . |
| . . L . . . . # # # # 0 . L . |
| . . . . # . 0 . L . . 0 . L . |
| . . # . . L . . . 0 . . L . # |
| L # . . . . L 2 . . L . 2 1 L |
| 1 # L . . 0 . L . # . . L 1 . |
+-----+
```

FIGURE 4 – Solution retournée par CPLEX

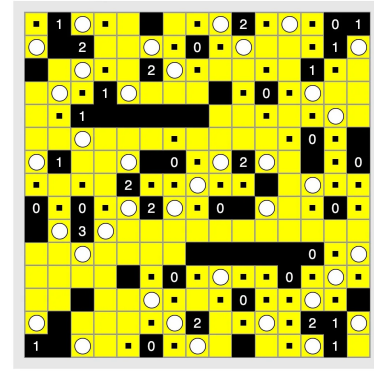


FIGURE 5 – Solution du site officiel

Instance 15×15 résolue en **0.024s**

6.2.2 Instance 20×20 — 0.025s

```
PS C:\Users\yarae\Desktop\JEUX et GRAPHES\JeuxGraph\Filling> julia
julia> testSolve()

=== Test cplexSolve ===
Optimal: true | Time: 0.025s

Solution:
+-----+
| . L . # L # L . 0 . # . . . L 1 . L . |
| . . L . # . . L . . # L . 0 . L 2 0 |
| . 0 2 L . . . # 2 L . 0 . # . . . # L |
| L . . . . . # . . . . . L . # . L 3 |
| . . 0 # 0 2 L . . 1 L . # L # . 0 . L |
| . L # 0 . L # . . . . L . . 1 . L . # |
| . # L . . 0 . L . . . # . L . # . L |
| . L . # . . L 3 L 1 . # . L 2 . L 1 # |
| . # . L . # L . . . . L 3 # . 0 . |
| . # . . L 1 # . . . . . L . 0 . L |
| . L . # . . . . . L # 1 . . . # . |
| L 2 . . # . . . . L 1 . L # . L . 0 # |
| # 1 . . # 1 . # . 0 . . 0 . # . L |
| . L # . L . 0 . L . . . 0 . L . 0 1 |
| . # . L . 0 . . . . L . 1 L . # . . |
| L . # . # . . . . 0 . . # # 2 L . |
| . # . # L 2 L . . . . . 0 . . . L . |
| L 2 L . . . # 1 L . 0 # . . L . 2 # . |
| # 0 . . L 1 L # . . . L 1 . L . . |
| L . . # . L . # . 0 . L # 0 . # L . |
+-----+
```

FIGURE 6 – Solution retournée par CPLEX

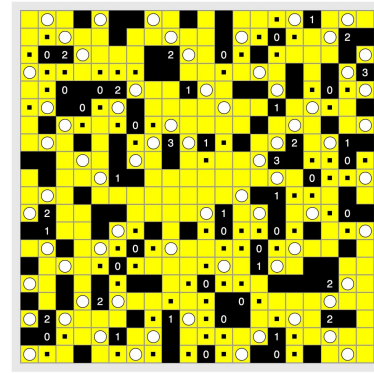


FIGURE 7 – Solution du site officiel

Instance 20×20 résolue en **0.025s**

6.2.3 Instance 35×35 — 0.069s

[illegible]

FIGURE 8 – Solution retournée par CPLEX



FIGURE 9 – Solution du site officiel

Instance 35×35 résolue en **0.069s**

6.2.4 Instance 50×50 — 0.038s

[illegible]

FIGURE 10 – Solution retournée par CPLEX



FIGURE 11 – Solution du
site officiel

Instance 50×50 résolue en **0.038s**

6.3 Comparaison des temps de résolution

Taille	Nombre de cases	Solution Trouvée ?	Temps (s)
15×15	225	✓	0.024
20×20	400	✓	0.025
35×35	1225	✓	0.069
50×50	2500	✓	0.038

TABLE 2 – Résolution CPLEX sur des instances réelles de tailles croissantes

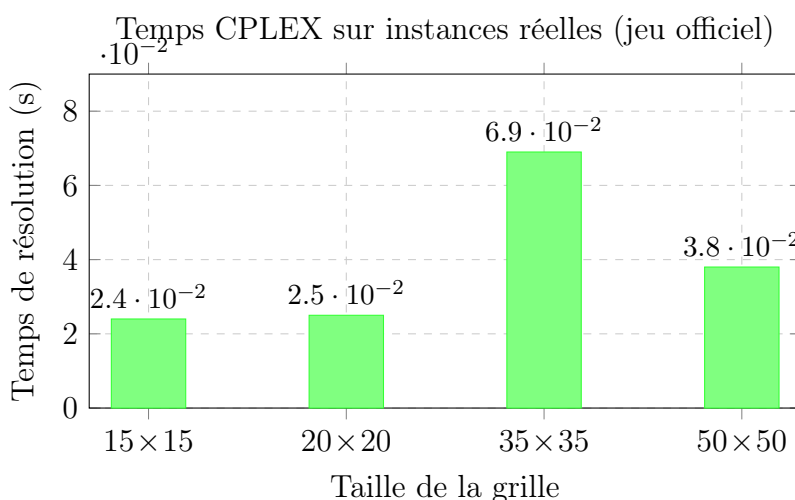


FIGURE 12 – Temps de résolution CPLEX en fonction de la taille

6.4 Analyse

Toutes les instances réelles sont résolues avec succès, y compris la grille 50×50 (2500 cases) en seulement **0.038s**. Les solutions retournées par CPLEX coïncident avec celles du site officiel, ce qui valide la correction de notre modèle.

La taille n'est pas le facteur limitant. Le solveur passe de 15×15 à 50×50 sans difficulté. La 50×50 est même plus rapide que la 35×35 (0.038s contre 0.069s), ce qui montre que la taille seule ne détermine pas la complexité.

La densité des contraintes est le vrai critère. C'est la disposition des cases noires numérotées qui conditionne le temps de calcul. Une instance de grande taille avec des contraintes

bien espacées peut être plus simple à résoudre qu’une plus petite avec des contraintes denses et imbriquées.

Conclusion. Notre modèle PLNE est efficace et scalable : toutes les instances testées sont résolues en moins de **0.07s**, quelle que soit la taille, jusqu’à 50×50 .

6.5 Impact de la densité de cases noires sur le temps de résolution

Pour analyser l’influence de la structure de la grille sur les performances du solveur, nous avons testé quatre instances 14×14 générées avec des densités croissantes de cases noires : 20%, 30%, 50% et 70%. Toutes les instances sont résolues.

Densité de cases noires	Cases blanches	Solution trouvée ?	Temps (s)
20%	158 / 196	✓	1.255
30%	137 / 196	✓	0.013
50%	98 / 196	✓	0.015
70%	59 / 196	✓	0.014

TABLE 3 – Temps de résolution CPLEX sur une grille 14×14 selon la densité de cases noires

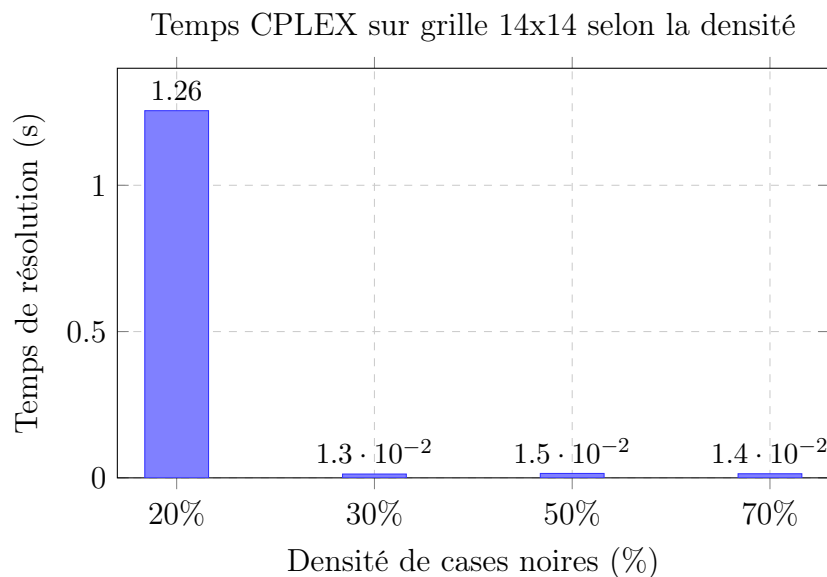


FIGURE 13 – Impact de la densité de cases noires sur le temps de résolution (grille 14×14)

Analyse. Le résultat le plus frappant est l'écart entre 20% (1.255s) et les autres densités (≤ 0.015 s). Ce phénomène s'explique de la façon suivante :

- **Faible densité (20%)** : peu de cases noires signifie beaucoup de cases blanches et donc un très grand nombre de placements de lampes possibles. Le système de contraintes est peu restrictif, ce qui oblige CPLEX à explorer un espace de recherche beaucoup plus large avant de trouver une solution valide.
- **Densité élevée (30–70%)** : les cases noires réduisent les segments visibles et multiplient les contraintes d'éclairage et de non-visibilité. Le problème est plus contraint, ce qui guide CPLEX vers la solution rapidement. On observe que 30%, 50% et 70% donnent des temps quasi-identiques (0.013–0.015s) : au-delà d'un certain seuil, la densité supplémentaire n'apporte plus de gain significatif.

Ce résultat illustre un principe général en optimisation combinatoire : un problème **plus contraint** est souvent **plus facile** à résoudre pour un solveur exact, car l'espace des solutions admissibles est réduit.

7 Conclusion

Light Up se modélise naturellement en PLNE avec trois types de contraintes binaires. L'implémentation en Julia avec JuMP et CPLEX permet une résolution exacte et rapide. La clé du code est la fonction `visibleFrom`, utilisée à la fois pour l'éclairage et pour interdire la visibilité entre lampes. La génération aléatoire produit beaucoup d'instances infaisables, ce qui est attendu pour des grilles non construites manuellement. En revanche, les tests sur de vraies instances confirment que c'est bien la **faisabilité**, et non la taille, qui constitue le vrai critère de difficulté.

Références

- [1] Simon Tatham's Portable Puzzle Collection. *Filling*. Disponible en ligne : <https://www.chiark.greenend.org.uk/~sgtatham/puzzles/>